Integrazione Revolut Terminal

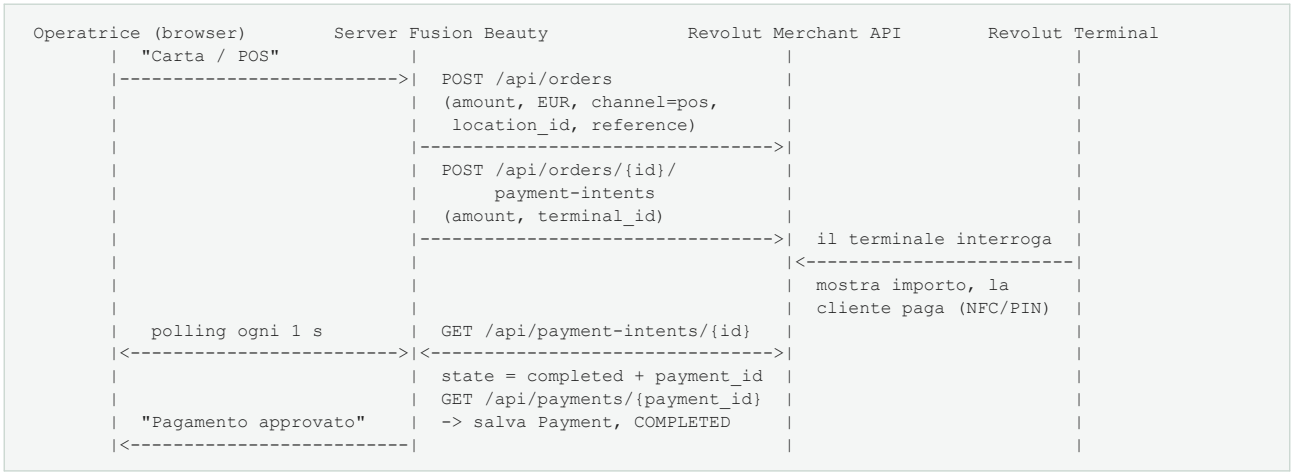
Analisi tecnica per Fusion Beauty (gestionale centri estetici) · 17 ottobre 2026

1. In sintesi

- **Si può fare, ed è un'integrazione pulita.** Dal 1° dicembre 2025 Revolut espone nella Merchant API le **push payments to Revolut Terminal**: il nostro server invia l'importo al terminale fisico via REST, senza SDK, Bluetooth o cavi. Il terminale mostra l'importo, la cliente appoggia la carta, noi leggiamo l'esito.
- **Flusso:** creare un *ordine* (canale `pos` + `location_id`) → creare un *payment intent* verso un `terminal_id` → interrogare lo stato ogni secondo → con lo stato `completed` registrare il pagamento nel gestionale.
- **Lato centro estetico:** serve un account Revolut Business con merchant account, un Revolut Terminal (189 € + IVA, Lite 139 € + IVA), una *location fisica* e la modalità **Pay at Counter** attiva sul terminale.
- **Lato codice:** oggi il pulsante "Carta / POS" registra subito un pagamento CARD (simulazione). Va sostituito da un flusso asincrono con stati *in attesa → in corso → approvato/rifiutato/annullato*. Servono un nuovo modello per le transazioni al terminale, un client API Revolut, 3 server action, un webhook e una pagina impostazioni. Stima: **circa 6–9 giorni/uomo** fino al pilota in un centro.
- **Attenzione fiscale (Italia):** il Revolut Terminal non è un registratore telematico. Il centro deve comunque emettere il documento commerciale e, dal 2026, **abbinare il POS al registratore telematico** sul portale Fatture e Corrispettivi (sanzione 1.000–4.000 €). L'integrazione non cambia questo obbligo.
- **Punto bloccante da chiarire con Revolut:** come testare con un terminale fisico (la sandbox non ha terminali documentati) e se esiste un programma partner per software multi-tenant come il nostro.

2. Come funziona l'integrazione push

È un'integrazione **server-to-server**. Il terminale e il nostro server non si parlano mai direttamente: entrambi parlano con Revolut. Il terminale in modalità Pay at Counter resta in attesa e interroga Revolut per i payment intent assegnati alla sua location; il nostro server crea l'intent e ne segue lo stato.



Ambienti

Ambiente	URL base	Note
Produzione	https://merchant.revolut.com/api	Secret key di produzione, terminale reale
Sandbox	https://sandbox-merchant.revolut.com/api	Chiavi sandbox; supporto ai terminali da verificare

Ogni chiamata richiede due header: `Authorization: Bearer <secret key>` e `Revolut-Api-Version: <data>`. Gli endpoint dei terminali compaiono nelle versioni documentate 2025-10-16 e 2025-12-04; conviene fissare **2025-12-04** (o successiva) e testarla.

3. Endpoint necessari

Metodo e percorso	A cosa serve	Campi chiave
GET /api/terminals?operation_mode=pos&location_id=…	Elenco terminali in Pay at Counter, per la pagina impostazioni e per scegliere il terminale	id, name, type, serial_number, online, battery_level, last_online_at
GET /api/locations?type=physical	Trovare la location del centro (senza *type* restituisce solo quelle online)	id
POST /api/locations	Creare la location fisica se manca	type: physical
POST /api/orders	Creare l'ordine da incassare	amount (centesimi), currency EUR, channel: pos, location_id, merchant_order_data.reference, metadata, line_items
POST /api/orders/{order_id}/payment-intents	Inviare l'importo al terminale	amount (deve coincidere con l'ordine), terminal_id
GET /api/payment-intents/{id}	Polling dello stato (ogni 1 s, timeout ~60 s)	state, payment_id
POST /api/payment-intents/{id}/cancel	Annullare dal gestionale (solo se ancora *pending*)	-
GET /api/payments/{payment_id}	Esito definitivo del pagamento	state, decline_reason
POST /api/orders/{order_id}/refund	Rimborso totale o parziale	amount
POST /api/webhooks	Notifiche di backup (ORDER_COMPLETED, ORDER_CANCELLED, ORDER_FAILED, ORDER_PAYMENT_DECLINED…)	url, events → signing_secret

Stati del payment intent

Stato Revolut	Significato	Cosa mostriamo all'operatrice
pending	Inviato al terminale, la cliente non ha ancora iniziato	"In attesa sul terminale…" + pulsante Annulla
processing	La cliente sta pagando (carta, PIN)	"Pagamento in corso…" (non annullabile)
completed	Autorizzazione conclusa, c'è un payment_id	Verifica del pagamento poi "Approvato"
cancelled	Annullato da noi o dalla cliente sul terminale	"Annullato" + Riprova / Contanti
failed	Errore tecnico (timeout, terminale offline, rete)	"Non riuscito" + Riprova

Importante: *completed* sul payment intent significa che il flusso sul terminale è finito, non che il pagamento sia andato a buon fine. Prima di segnare l'appuntamento come pagato bisogna leggere `GET /api/payments/{payment_id}` e accettare solo stati come `authorised`, `captured` o `completed`. Con `declined` o `failed` mostriamo il motivo (`decline_reason`, ad esempio insufficient_funds o expired_card).

4. Prerequisiti per ogni centro estetico

#	Cosa	Chi lo fa
1	Conto Revolut Business con **merchant account** attivo (accettazione pagamenti)	Titolare del centro
2	Acquisto e attivazione del Revolut Terminal	Titolare
3	Location **fisica** creata e terminale assegnato a quella location	Titolare (app) o noi via API
4	Sul terminale: **More** → **Settings** → **Checkout** → **Pay at Counter** attivo	Titolare / installazione assistita
5	Generazione della **Secret API key** (Business → APIs → Merchant API) e inserimento in Fusion Beauty	Titolare, guidato da noi
6	Abbinamento POS ↔ registratore telematico sul portale Agenzia Entrate	Titolare o commercialista

Con Pay at Counter attivo il terminale resta in uno stato passivo, in attesa delle richieste dal gestionale. Va chiarito con Revolut se il terminale può ancora fare incassi manuali mentre è in questa modalità (utile per vendite al banco senza appuntamento).

5. Stato attuale del codice

- `src/app/dashboard/payments/payment-dialog.tsx`: il dialog Incasso ha due pulsanti, "Carta / POS" e "Contanti". Entrambi chiamano subito `registerPayment`: per la carta non c'è nessuna comunicazione con un POS.
- `src/app/dashboard/payments/actions.ts`: `registerPayment` in una transazione crea o aggiorna il `Payment` (uno per appuntamento, `appointmentId @unique`) e mette l'appuntamento in COMPLETED.
- `prisma/schema.prisma`: `Payment` non ha campi per il riferimento esterno (ID ordine o pagamento Revolut, stato, provider). `TenantIntegration` ha già `credentials Json` e `@@unique([tenantId, type])`, ma `IntegrationType` prevede solo GOOGLE_CALENDAR e ICAL_URL.
- Le credenziali in `credentials` sono salvate in chiaro: per una secret key di pagamento non va bene, serve la cifratura.
- C'è già un worker BullMQ con Redis (`src/workers/index.ts`), ma ha solo una coda di esempio. Può servire per riconciliazione e controlli in background.
- Le API route stanno in `src/app/api/*`: il webhook si aggiunge lì.

6. Design proposto

6.1 Modello dati

Il Payment resta il **registro dell'incasso riuscito**. Ogni tentativo al terminale viene tracciato in un nuovo modello, così un tentativo rifiutato o annullato non crea un Payment, abbiamo l'audit di tutti i tentativi e possiamo bloccare i doppi addebiti.

```
enum IntegrationType { GOOGLE_CALENDAR  ICAL_URL  REVOLUT_MERCHANT }   // + nuovo valore

enum TerminalPaymentStatus { CREATED PENDING PROCESSING SUCCEEDED DECLINED CANCELLED FAILED }

model TerminalPayment {
  id               String                @id @default(cuid())
  tenantId         String
  appointmentId    String
  amount           Decimal               @db.Decimal(10, 2)
  status           TerminalPaymentStatus @default(CREATED)
  revolutOrderId   String?               @unique
  revolutIntentId  String?               @unique
  revolutPaymentId String?
  terminalId       String
  declineReason    String?
  paymentId        String?               @unique   // Payment creato all'esito positivo
  createdById      String?
  createdAt        DateTime              @default(now())
  updatedAt        DateTime              @updatedAt
  @@index([tenantId, appointmentId])
}

model Payment {            // campi aggiunti
  provider          String?   // "revolut_terminal" | null (manuale)
  externalPaymentId String?   @unique
}

model User { revolutTerminalId String? }  // terminale preferito dell'operatrice (multi-postazione)
```

La configurazione del centro (location_id, terminale predefinito, signing secret del webhook, secret key cifrata) va in `TenantIntegration` con type REVOLUT_MERCHANT. La secret key va cifrata con AES-256-GCM, con una master key in variabile d'ambiente.

6.2 Server action (src/app/dashboard/payments/terminal-actions.ts)

Action	Comportamento
startTerminalPayment({ appointmentId, amount, notes, terminalId? })	Verifica tenant e appuntamento; blocca se c'è già un TerminalPayment attivo (PENDING o PROCESSING) per l'appuntamento; crea il TerminalPayment; chiama POST /orders (reference = appointmentId, metadata con tenantId e terminalPaymentId) e poi POST /payment-intents; restituisce l'id interno.
pollTerminalPayment(id)	Chiamata dal client ogni ~1 s. Legge l'intent da Revolut e aggiorna lo stato. Con completed legge il payment e, se è riuscito, in un'unica transazione crea il Payment (method CARD, provider, externalPaymentId), collega il TerminalPayment e mette l'appuntamento in COMPLETED. Deve essere idempotente: può essere chiamata più volte, e anche il webhook può arrivare per primo.
cancelTerminalPayment(id)	POST /payment-intents/{id}/cancel, possibile solo se lo stato è pending.

La finalizzazione (creazione del Payment) va in un'unica funzione condivisa da polling, webhook e job di riconciliazione. Il vincolo `@unique` su `externalPaymentId` e `appointmentId` protegge dalle scritture doppie.

6.3 Client Revolut (src/lib/revolut/merchant.ts)

```
import "server-only";

const BASE = process.env.REVOLUT_ENV === "sandbox"
  ? "https://sandbox-merchant.revolut.com/api"
  : "https://merchant.revolut.com/api";
const API_VERSION = "2025-12-04";

async function revolut<T>(secretKey: string, path: string, init: RequestInit = {}): Promise<T> {
  const res = await fetch(`${BASE}${path}`, {
    ...init,
    headers: {
      Authorization: `Bearer ${secretKey}`,
      "Revolut-Api-Version": API_VERSION,
      "Content-Type": "application/json",
      ...init.headers,
    },
    cache: "no-store",
  });
  if (!res.ok) throw new RevolutError(res.status, await res.text());
  return res.json() as Promise<T>;
}

export const toMinor = (euro: number) => Math.round(euro * 100);   // 70.34 -> 7034

export const createPosOrder = (key: string, o: { amount: number; locationId: string;
  reference: string; description: string; metadata: Record<string, string> }) =>
  revolut<{ id: string }>(key, "/orders", { method: "POST", body: JSON.stringify({
    amount: toMinor(o.amount), currency: "EUR", channel: "pos", location_id: o.locationId,
    description: o.description, merchant_order_data: { reference: o.reference },
    metadata: o.metadata,
  }) });

export const pushToTerminal = (key: string, orderId: string, amount: number, terminalId: string) =>
  revolut<PaymentIntent>(key, `/orders/${orderId}/payment-intents`, { method: "POST",
    body: JSON.stringify({ amount: toMinor(amount), terminal_id: terminalId }) });
```

6.4 Interfaccia (dialog Incasso)

- Il pulsante "Carta / POS" apre lo stato **In attesa sul terminale** con nome del terminale, importo e pulsante **Annulla**. Durante *processing* mostra "Pagamento in corso…" e il pulsante Annulla è disattivato.
- Esiti: **Approvato** (schermata di conferma come oggi), **Rifiutato** con il motivo tradotto in italiano, **Annullato** o **Non riuscito**, con le azioni Riprova o Incassa in contanti.
- Se il dialog viene chiuso durante l'attesa, il tentativo resta attivo: la lista "Da incassare" mostra un badge "Sul terminale" e la finalizzazione arriva via webhook o riconciliazione.
- Se il terminale è offline (letto da GET /terminals) lo segnaliamo prima di inviare. Senza integrazione configurata il pulsante mantiene il comportamento attuale (registrazione manuale CARD), quindi la demo continua a funzionare.
- Impostazioni (solo admin): inserimento della secret key, scelta della location, elenco terminali con stato e batteria, terminale predefinito e terminale per operatrice, pulsante "Invia 0,01 € di prova".

6.5 Webhook e riconciliazione

- Route `POST /api/webhooks/revolut/[tenantId]`, registrata con POST /api/webhooks quando l'admin salva l'integrazione. Eventi: ORDER_COMPLETED, ORDER_AUTHORISED, ORDER_CANCELLED, ORDER_FAILED, ORDER_PAYMENT_DECLINED, ORDER_PAYMENT_FAILED.
- Verifica della firma HMAC-SHA256 con il `signing_secret` ricevuto alla registrazione (header `Revolut-Signature` e timestamp) e rifiuto degli eventi vecchi. Lo schema esatto della firma va confermato sulla guida ufficiale.
- Gli eventi possono arrivare **fuori ordine** (lo dice la documentazione): il webhook non decide lo stato, rilegge l'ordine via API e chiama la stessa funzione di finalizzazione.

- Non esistono eventi webhook specifici per i payment intent: il polling resta il canale principale e il webhook copre i casi in cui il browser si è chiuso.
- Job BullMQ ogni 2–5 minuti: riprende i TerminalPayment PENDING o PROCESSING più vecchi di 2 minuti e li riallinea.

6.6 Rimborsi

Pulsante "Rimborsa" su un Payment con provider revolut_terminal, solo per admin: chiama POST /orders/{id}/refund e salva il rimborso (serve un modello Refund o un campo `refundedAmount`). Statistiche e report devono sottrarre i rimborsi dagli incassi.

7. Sicurezza e multi-tenant

- Ogni centro ha il **proprio** account Revolut e la propria secret key: i fondi vanno direttamente al centro, Fusion Beauty non tocca mai il denaro (nessuna licenza di pagamento necessaria).
- La secret key dà accesso completo al merchant account (ordini, rimborsi). Va cifrata a riposo, usata solo lato server (`server-only`), mai inviata al client né scritta nei log. La modifica è riservata al ruolo ADMIN.
- Ogni server action filtra per `tenantId` (come avviene già oggi) e verifica che il terminal_id appartenga al terminale configurato per il tenant.
- Rate limit e un solo tentativo attivo per appuntamento, per evitare doppi addebiti causati da doppi click.
- **Flag "Pagamento privato":** nasconde l'incasso alle operatrici nel gestionale. Un pagamento con carta resta comunque tracciato da Revolut e, con l'abbinamento POS–RT, comunicato all'Agenzia delle Entrate. Conviene spiegarlo chiaramente nell'interfaccia, per non dare l'impressione che il flag abbia effetti fuori dal gestionale.

8. Aspetti fiscali in Italia

- Il Revolut Terminal incassa ma **non emette il documento commerciale**: il centro continua a usare il registratore telematico, o la procedura web dell'Agenzia, per ogni corrispettivo.
- Dal 2026 è obbligatorio l'**abbinamento logico POS ↔ RT** nel portale "Fatture e Corrispettivi", sezione "Gestione collegamenti" (non serve nessun cavo). Per un POS nuovo il termine va dal 6° giorno all'ultimo giorno lavorativo del secondo mese dopo l'attivazione. Per i POS attivi a gennaio 2026 la scadenza era il 20 aprile 2026. La sanzione va da 1.000 a 4.000 €. I parrucchieri e gli estetisti sono espressamente inclusi.
- **Opportunità di prodotto (fase successiva):** integrare un RT cloud o un servizio di documento commerciale online, così da emettere lo scontrino in automatico dopo l'incasso. Il flusso completo diventerebbe incasso più corrispettivo in un solo tocco, un punto di forza commerciale.

9. Rischi e punti aperti

Tema	Rischio / incertezza	Azione
Test	Non è documentato come usare un terminale fisico in sandbox	Chiedere a Revolut; in alternativa test in produzione con importi da 0,01–1 € e rimborso
Campo *channel*	Richiesto dall'endpoint payment-intent ma assente dallo schema pubblico di create-order	Verificarlo con la prima chiamata reale e leggere il 422
Versione API	Gli endpoint terminali compaiono solo nelle versioni recenti	Fissare Revolut-Api-Version 2025-12-04 e testare
Programma partner	Ogni centro deve generare a mano una secret key, con possibili problemi di onboarding	Chiedere a Revolut se esiste OAuth o un programma partner per POS di terze parti
Uso misto	In Pay at Counter il terminale è passivo: non è chiaro se accetti anche vendite manuali	Verificare con Revolut o sul dispositivo
Timeout	La cliente si allontana e l'intent resta pending	Timeout lato UI di 90 s con annullamento automatico; job di riconciliazione
Rete	Il server perde la risposta di create-intent e l'utente riprova, con rischio di doppio intent	Prima di riprovare controllare gli intent aperti; un tentativo attivo per appuntamento
Commissioni	Le tariffe in presenza dipendono dal piano e dal tipo di carta	Verificare il listino Revolut aggiornato prima della proposta commerciale

10. Piano di lavoro e stima

Fase	Attività	Giorni
0. Preliminari	Account Revolut Business di test, terminale, domande aperte a Revolut, prima chiamata manuale con curl	0,5–1
1. Fondamenta	Migrazione Prisma (TerminalPayment, campi Payment, enum), cifratura credenziali, client API tipizzato	1–1,5
2. Impostazioni	Pagina admin: secret key, location, elenco terminali, terminale per operatrice, pagamento di prova	1
3. Flusso incasso	Server action start/poll/cancel, finalizzazione idempotente, nuovi stati nel dialog Incasso	1,5–2
4. Robustezza	Webhook con verifica firma, job BullMQ di riconciliazione, badge "Sul terminale"	1–1,5
5. Rimborsi e report	Rimborso da scheda pagamento, adeguamento statistiche, report e CSV	0,5–1
6. Pilota	Test end-to-end con un terminale reale in un centro, rifinitura dei messaggi d'errore	1
Totale		**6,5–9**

Per la demo è sufficiente la fase 3 con un account reale. Il comportamento attuale resta come fallback quando l'integrazione non è configurata.

Prossimi passi consigliati

- Aprire o usare un account Revolut Business con merchant account e ordinare un terminale per lo sviluppo.
- Scrivere al supporto sviluppatori Revolut le domande della sezione 9: sandbox con terminali, programma partner, campo channel, uso misto.
- Fare una prova con curl: GET /terminals, POST /orders, POST /payment-intents con 0,01 €, poi il rimborso.
- Avviare le fasi 1–3.

Fonti

Revolut Developer – Terminals API: developer.revolut.com/docs/api/merchant/tags/tag-terminals

Create a payment intent: developer.revolut.com/docs/api/merchant/operations/create-payment-intent

Retrieve / Cancel a payment intent: developer.revolut.com/docs/api/merchant/operations/retrieve-payment-intent, …/cancel-payment-intent

Retrieve terminal list: developer.revolut.com/docs/api/merchant/operations/retrieve-terminal-list

Create an order, Retrieve payment details, Refund an order, Locations, Webhooks: developer.revolut.com/docs/api/merchant

Guida Push payments to Revolut Terminal: developer.revolut.com/docs/guides/in-person-payments/revolut-terminal/push-payments

Annuncio (1 dic 2025): developer.revolut.com/updates/2025/12/01/push-payments-to-terminal

Revolut Help – integrazione con POS di terze parti e Pay at Counter: help.revolut.com (merchant accounts › Revolut Terminal)

Prezzi del terminale: revolut.com/it-IT/business/revolut-terminal

Agenzia delle Entrate – comunicato stampa del 17 aprile 2026 sull'abbinamento RT–POS: agenziaentrate.gov.it

Approfondimenti sull'obbligo POS–RT: artigiani.tn.it, fiscoetasse.com, beautycheck.it

Le specifiche API sono state lette dalla documentazione ufficiale nella versione disponibile il 17/10/2026. Prima dello sviluppo vanno ricontrollati i prezzi e lo schema della firma dei webhook.